

GCP for SQL Server Professionals — the Mapping Card

The big mental-model card: your SQL Server world on the left, the GCP equivalent on the right. Read it once before Day 1, keep it open during labs, and let the right-hand column become reflexive by Tuesday afternoon.

One caution before the table: these are *mappings*, not identities. A GCP project behaves like an instance in the ways that matter to you (boundary of access and billing) and differently in ways that don't (nobody patches it). Where the difference bites, the notes column says so.

Compute & structure

Your SQL Server world	GCP world	Notes
SQL Server instance	GCP project	The top-level container: resources, permissions, and billing live at the project boundary. Nothing to install, patch, or fail over.
Database	BigQuery dataset	Container for tables and views. You choose its location (region/multi-region) at creation and can't change it later — decide where your warehouse lives once, deliberately.
schema.table	dataset.table	BigQuery has no schema layer. The fully qualified name is `project.dataset.table` — three parts, backticks.
Sizing the instance (cores, RAM)	Nothing to size	BigQuery is serverless: compute is automatic per query. Capacity planning only re-enters if you later choose flat-rate slots instead of on-demand pricing.

Storage

Your SQL Server world	GCP world	Notes
MDF/NDF data files	BigQuery managed storage	You never see, place, or manage storage files —

Your SQL Server world	GCP world	Notes
File shares / SFTP drops	GCS buckets (Cloud Storage)	you create tables, storage follows. Object storage. "Folders" are naming conventions in object keys, not real directories — but the Hive layout (county=X/period=YYYY-MM/) buys you real partition pruning.
Staging share for extracts	A GCS bucket (your Bronze layer)	Files land once, stay immutable, and are queryable in place via external/BigLake/Iceberg tables. No staging-server disk to babysit.
Backup & restore	Time travel + table snapshots	Query any table as it was up to 7 days ago (default window); snapshots for longer. GCS versioning covers the bucket side.

Security

Your SQL Server world	GCP world	Notes
Logins & users	Google identities + IAM	Access is granted to people, groups, or service accounts — on projects, datasets, or individual tables.
SQL Agent proxies / job accounts	Service accounts	Non-human identities for pipelines. The least-privilege discipline you already apply transfers one-for-one: a refresh identity that may only read the Gold dataset and run queries.
db_datareader / db_datawriter	IAM roles (e.g., BigQuery Data Viewer, Job User)	Roles bundle permissions; scope them as tight as you scope database roles today.
Row/column-level	Row access policies,	Exists in BigQuery —

Your SQL Server world	GCP world	Notes
security, dynamic masking	column-level security, policy tags	covered in the Day-2 governance block.

Operations & orchestration

Your SQL Server world	GCP world	Notes
SSIS packages	Dataform (ELT inside BigQuery) / Dataflow (heavy or streaming transforms)	Most of your transforms become plain SQL in Dataform models. Dataflow is the escape hatch when SQL genuinely isn't enough.
SQL Server Agent jobs	Cloud Scheduler / scheduled queries	A SQL statement on a schedule — you can build one with a few clicks, and you will on Day 1.
Linked servers / OPENQUERY	Federated queries (connection resource + EXTERNAL_QUERY) / BigLake for files	The connection resource is the linked-server definition; EXTERNAL_QUERY is OPENQUERY — including the "inner SQL is the remote dialect" trap.
Replication / log shipping / CDC	Datastream	Log-based CDC from SQL Server (and Oracle/MySQL/PostgreSQL) into BigQuery: backfill plus continuous changes, no triggers, no nightly window.
Windows Task Scheduler	Cloud Scheduler	Managed cron for anything on a timer.
SSIS projects in source control	Dataform repositories in git	.sqlx files, branches, pull requests, release configurations — the shape of the CI/CD pipelines you already run.
Agent job history / DMVs	Job history, INFORMATION_SCHEMA views, Cloud	Every query and load is a job with metadata you can query in SQL.

Monitoring

Instincts that transfer

You don't need new engineering judgment for this platform — you need to point your existing judgment at new consoles. Seven instincts, unchanged:

1. **Your CI/CD discipline is exactly how Dataform releases work.** Models are .sqlx files in git; you review changes in PRs; a release configuration promotes a git commit to run on a schedule. The pipeline rigor your team built for SQL Server deployments is the correct mental model, and most teams new to GCP don't have it — you do.
2. **You never trusted 'Suggest Types' in SSIS — don't trust autodetect here.** Autodetect samples the start of a file; a dirty value deeper in gets either a loud failure or a silently wrong schema (and the silent one costs you days). For anything recurring: pin a schema JSON in source control and name your null markers (--null_marker='N/A'). You do this dance on Day 1.
3. **Jobs should be idempotent and re-runnable.** Same rule, new syntax: overwrite partitions (WRITE_TRUNCATE), use incremental models keyed on a watermark, and design so that re-running last Tuesday is a non-event.
4. **Least-privilege service identities.** A job gets a service account scoped to exactly what it reads and writes — the proxy-account discipline, one-for-one.
5. **Environment separation.** Dev/test/prod become separate GCP projects (or datasets), with IAM doing the walling-off you used to do with separate instances.
6. **Estimate before you execute.** You read execution plans before running something big in production; here you read the **bytes-processed estimate** next to the RUN button before pressing it. Same instinct, cheaper mistakes.
7. **Monitor freshness, not job success.** A replication stream has a freshness metric instead of a 2 AM job that either mailed you or didn't. The M&O question shifts from "did the job finish?" to "how stale is the data?" — a better question, and one you already know how to operationalize.

Where this card gets used

- Day 1, 09:25 — the building-blocks block walks the compute & storage rows
- Day 1, 11:10 — the replication row becomes your extract-killer
- Day 1, 16:05 — the orchestration rows become your automation map
- Day 2, 14:00 — the security rows become governance across the medallion
- Always: the [glossary](#) defines any term above in one line

Part of the Iowa DOM DoIT Data Engineering student pack — see ../README.md.